計算物理学II

第8講：Pythonによる数値計算プログラミング入門（4）

秋山 進一郎

2024年11月25日

授業日の確認

- 第1講：10月4日（金）
- 第2講：10月11日（金）
- 第3講：10月18日（金）
- 第4講：10月25日（金）★
- 第5講：11月1日（金）
- 第6講：11月15日（金）★
- 第7講：11月22日（金）
- **第8講：11月25日（月）★**
- 第9講：12月6日（金）
- 第10講：12月13日（金）★

・★の付いた授業にてレポート課題を配布予定

今日の授業の目標

- Pythonでのファイル操作に慣れる
- matplotlibを使ったグラフの作成
- 終わった人は第3回レポート課題へ

毎回の授業の流れ

“compphy2”に移動し，“git branch”で確認

↓

“git fetch”と“git merge”で演習資料を入手

↓

“lecture*”に移動し，
“lecture_material_*.pdf”を開いて演習開始

本日の演習内容

- Pythonでのファイル操作に慣れる
  - データをデータファイルとして入手し，プログラムに入力する
  - 入力されたデータに対して様々な処理が行えることを理解する
- matplotlib.pyplotを使ってデータの可視化ができるようになる
  - いくつかの図を書いてみる

ブランチを切る

- 原則, Gitではmainブランチで作業しない
  - “exercise”という名前のブランチを作成（①）
  - mainからexerciseへの切り替え（②）
  - 自分がいるブランチの確認（③）

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch exercise   ①
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git switch exercise   ②
Switched to branch 'exercise'
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch            ③
* exercise
  main
```

ファイルの入出力の基本

・データファイル(.txt, .dat, .csv, .jsonファイル等)をPythonで扱う手続き

① openを使ってファイルを開く

② ファイル中のデータに対する処理を行う

③ 処理が終わったらcloseを使ってファイルを閉じる

ファイルへの書き込みをやってみよう

- 右のようなコードを書き，“ex_write.py”の名前で保存しよう

```
1    fileRef = open("test.txt","w")
2    fileRef.write("Hello World\n")
3    fileRef.close()
```

- コードの中身は以下の通り
  - 1行目：text.txtという名前のファイルを開く(“w”は書き込みモードの意味)
    ファイルオブジェクトとしてfileRefがopenから返され，以降コード上では
    fileRefを介してデータの処理ができるようになる
  - 2行目：文字列を書き込む．最後の＼nは改行を与える
  - 3行目：ファイルを閉じる
- “ex_write.py”を実行してみよう
  - 実行後，lsで見ると“test.txt”ができているはずである
  - “test.txt”をcatで見てみよう(Hello Worldと書いてあれば成功)

ファイルへ追記してみよう

- 下のようなコードを書き，“ex_add.py”の名前で保存しよう

```
fileRef = open("test.txt","a")
fileRef.write("The second line is added\n")
fileRef.close()
```

- openの第二引数の“a”は追記モードの意味
  - すでに存在するファイル(test.txt)にデータを追記したい場合に使うモード
- “ex_add.py”を実行してみよう
  - 実行後，“test.txt”をcatで見てみよう(以下のようになっていればOK)

```
Hello World
The second line is added
```

書き込みモードを使う時の注意点

- すでに存在するファイルを書き込みモード(w)で開くと，そのファイルの内容は削除される
- 下のようなコードを書き，実行してみよう
  - すでに存在するtest.txtを書き込みモードで開き，その後閉じるだけのコード

```
fileRef = open("test.txt","w")
fileRef.close()
```

- 実行後，“test.txt”をcatで見てみよう
  - test.txtの内容が削除されていることが確認できればOK
- ファイルを書き込みモードで開くと，開いた瞬間にファイルの内容がリセットされることを忘れないように

ファイルを読み込んでみよう 1/2

- 下のようなコードを書き，“ex_read.py”の名前で保存しよう
- lecture8の中のdataディレクトリに“nobel_prize_2020s.txt”というファイルがある
  - このtxtファイルを“ex_read.py”が保存されいるディレクトリにcpしよう
  - catを使って，“nobel_prize_2020s.txt”の中身を見ておこう
- “ex_read.py”の2行目のfor文では，fileRefを1行ずつlineという変数に読み込み，3行目でlineを出力する．openの第二引数“r”は読み込みモード．読み込みモードがデフォルトなので省略してもよい

```
1   fileRef = open("nobel_prize_2020s.txt","r")
2   for line in fileRef:
3       print(line)
4   fileRef.close()
```

ファイルを読み込んでみよう 2/2

- “ex_read.py”を実行してみよう
  - 右のように1行毎に改行が入った出力が得られればOK
    - “ex_read.py”のような形で各行毎の読み込みを行うと，変数lineには改行コードが含まれるため
- line末尾の改行を削除するには，rstrip()を使うとよい
  - “ex_read.py”の3行目を以下のように修正して実行してみよう(改行が消えたことを確認しましょう)

```
1   fileRef = open("nobel_prize_2020s.txt","r")
2   for line in fileRef:
3       print(line.rstrip())
4   fileRef.close()
```

```
2020 Roger Penrose

2020 Reinhard Genzel

2020 Andrea Ghez

2021 Syukuro Manabe

2021 Klaus Hasselmann

2021 Giorgio Parisi

2022 Alain Aspect

2022 John Clauser

2022 Anton Zeilinger

2023 Anne L'Huillier

2023 Ferenc Krausz

2023 Pierre Agostini

2024 John Hopfield

2024 Geoffrey Hinton
```

with構文を使う

- with構文を使うことで，Pythonは自動でファイルを閉じてくれる
  - 明示的にclose()を書く必要がなくなる
- 下のようなコードを書き，実行してみよう
  - 先ほどの“ex_read.py”をwith構文を使って書き直したもの
  - コードブロックが終わった時に，ファイルが自動的にcloseされる
  - ファイルの入出力に際しては，with構文の使用が好ましい

```
1  with open("nobel_prize_2020s.txt","r") as fileRef:
2      for line in fileRef:
3          print(line.rstrip())
```

csvファイル

- csvファイルとは，コンマ(，)でデータが区切られたテキストファイルの総称
  - csv = comma-separated values
- 様々なオープンデータがcsvファイルで提供されている

過去の大学入試共通テストのデータを見てみよう

- lecture8/dataの中にscore.csvというファイルが格納してある
  - 大学入試共通テスト(旧センター試験も含む)のいくつかの科目について，受験者数と平均点が年度毎に収録されている
  - csvファイルの1行目はメタデータで，データに関する情報が記述されている
  - catでscore.csvを見て，ファイル内のデータ構造を確認しよう
    - 各行毎に年度が異なる
    - 各列に異なる項目のデータが記述されている
    - 収録科目は国語，数学IA，数学IIB，物理，化学，生物，地学，英語(reading)，英語(listening)．受験者数に続いて平均点が100点満点換算で記載されている

score.csvからデータを取得するコードを書こう 1/2

- 右のようなコードを書き，“score.py”という名前で保存しよう
  - 1~3行目は後で説明, 使用する
  - 5行目以下のコードブロックでscore.csvからデータを取得している
  - 6~9行目の空のリストは後で使う
  - 11行目では，score.csvから取得した各行について，split(“,”)によりカンマで分離したリストfileListを作っている

```
1   import matplotlib
2   matplotlib.use("Agg")
3   import matplotlib.pyplot as plt
4
5   with open("score.csv","r") as fileRef:
6       year = []
7       jpn = []
8       math_ia = []
9       math_iib = []
10      for line in fileRef:
11          fileList = line.split(",")
12          print(fileList)
```

- “score.py”を実行すると12行目により， score.csvの各行毎の要素から構成されたリストが出力されることを確認せよ
- type()を使い，fileListがstr型になっていることも確認しておこう

score.csvからデータを取得するコードを書こう 2/2

- 続いて“score.py”に右のようなコードを書き足そう
  - 13行目以下のコードは，これまでの演習で学んだ内容から理解できるはず
  - appendする時にfileListの要素の型をintやfloatに変換しておく
    - fileListはstr型になっていることに注意
- これで，score.csvの特定の列(0, 4, 6, 8列目)の要素のみからなるリストができる
- 以降では，これらのリストを使って図を描いていく

```
1   import matplotlib
2   matplotlib.use("Agg")
3   import matplotlib.pyplot as plt
4
5   with open("score.csv","r") as fileRef:
6       year = []
7       jpn = []
8       math_ia = []
9       math_iib = []
10      for line in fileRef:
11          fileList = line.split(",")
12          print(fileList)
13          if fileList[0] != "Year":
14              year.append(int(fileList[0]))
15              jpn.append(float(fileList[4]))
16              math_ia.append(float(fileList[6]))
17              math_iib.append(float(fileList[8]))
```

matplotlibを使ったPythonによるグラフ作成

- matplotlibはPythonでグラフ等を書く場合に便利なライブラリ
  - もちろんgnuplotを使っても良いが，Pythonでコードを書いているなら図もPythonで作れた方が便利でしょう
  - matplotlibの公式ドキュメントはここ

```
1    import matplotlib
2    matplotlib.use("Agg")
3    import matplotlib.pyplot as plt
```

- score.pyの1~3行目でmatplotlibをimportしていた
  - 3行目でmatplotlib.pyplotにpltという名前をつけてimportしている
  - 1, 2行目は対話型バックエンドが使えない場合に必要
    - 例えば，コード実行時に以下のようなErrorが出たら1,2行目を書けば良い

```
qt.qpa.plugin: Could not find the Qt platform plugin "xcb" in ""
This application failed to start because no Qt platform plugin could be initialized. Reinstalling the application may fix this problem.
```

matplotlibを使ってみよう 1/2

- score.pyにさらにコードを書き足しましょう
  - 19~29行目のコードを書きましょう
- 各行の意味は以下の通り
  - 19行目：描画領域を準備
  - 20行目：グラフの軸を準備．1つのグラフを書く場合は(1, 1, 1)とすればよい
  - 21~23行目：x軸にyear, y軸にjpn, math_ia, math_iibを使って折れ線グラフを作る．marker="o"でデータ点に丸シンボルを付す．labelで凡例の名前を設定
  - 24行目：凡例の表示

```
1  import matplotlib
2  matplotlib.use("Agg")
3  import matplotlib.pyplot as plt
4
5  with open("score.csv","r") as fileRef:
6      year = []
7      jpn = []
8      math_ia = []
9      math_iib = []
10     for line in fileRef:
11         fileList = line.split(",")
12         print(fileList)
13         if fileList[0] != "Year":
14             year.append(int(fileList[0]))
15             jpn.append(float(fileList[4]))
16             math_ia.append(float(fileList[6]))
17             math_iib.append(float(fileList[8]))
18
19 fig = plt.figure()
20 ax = fig.add_subplot(1, 1, 1)
21 ax.plot(year, jpn, marker = "o", label = "Japanese")
22 ax.plot(year, math_ia, marker = "o", label = "Math IA")
23 ax.plot(year, math_iib, marker = "o", label = "Math IIB")
24 ax.legend()
25 ax.set_xlim(2006, 2024)
26 ax.set_ylim(0, 100)
27 ax.set_xlabel("Year")
28 ax.set_ylabel("Average Score")
29 plt.savefig("score.pdf")
```

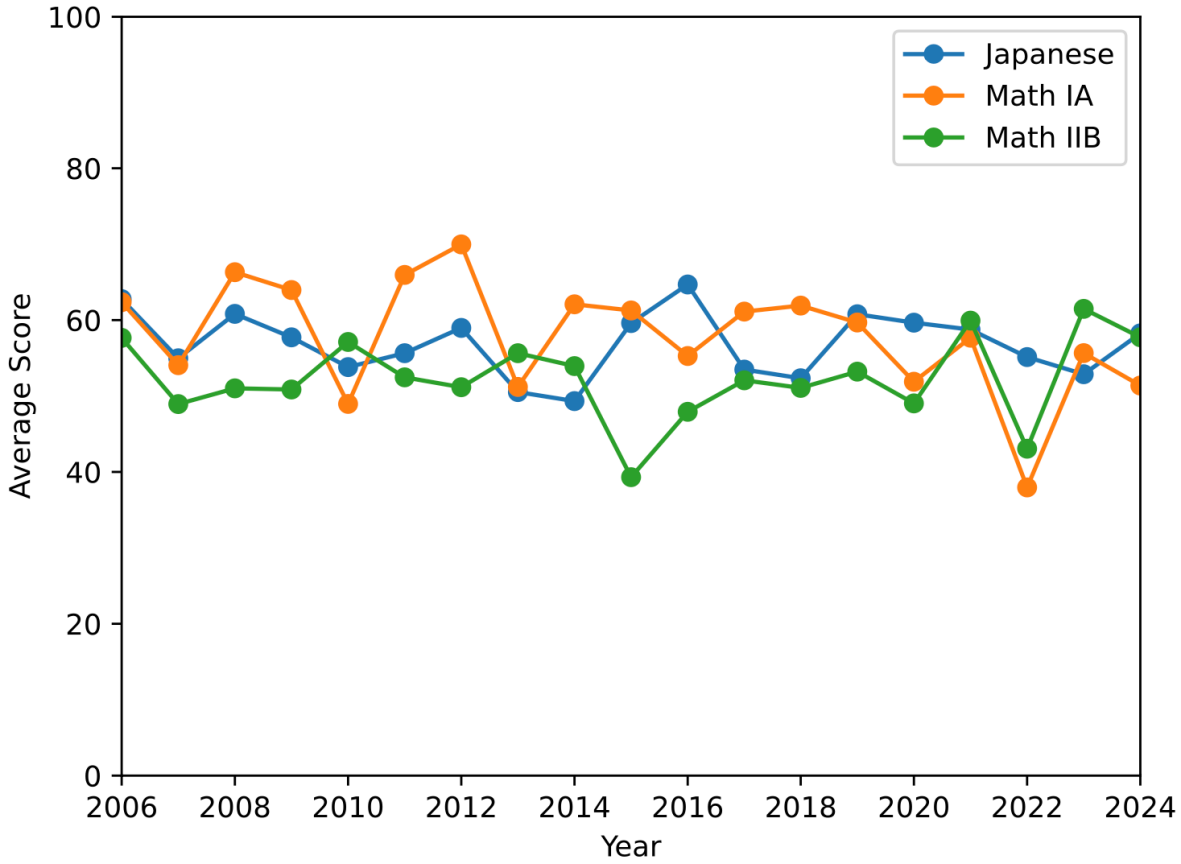
matplotlibを使ってみよう 2/2

- score.pyにさらにコードを書き足しましょう
  - 19~29行目のコードを書きましょう
- 各行の意味は以下の通り
  - 25, 26行目：x軸, y軸の範囲を設定
  - 27, 28行目：x軸, y軸のラベルを設定
  - 29行目：作成したグラフをscore.pdfとして保存

```
1  import matplotlib
2  matplotlib.use("Agg")
3  import matplotlib.pyplot as plt
4
5  with open("score.csv","r") as fileRef:
6      year = []
7      jpn = []
8      math_ia = []
9      math_iib = []
10     for line in fileRef:
11         fileList = line.split(",")
12         print(fileList)
13         if fileList[0] != "Year":
14             year.append(int(fileList[0]))
15             jpn.append(float(fileList[4]))
16             math_ia.append(float(fileList[6]))
17             math_iib.append(float(fileList[8]))
18
19 fig = plt.figure()
20 ax = fig.add_subplot(1, 1, 1)
21 ax.plot(year, jpn, marker = "o", label = "Japanese")
22 ax.plot(year, math_ia, marker = "o", label = "Math IA")
23 ax.plot(year, math_iib, marker = "o", label = "Math IIB")
24 ax.legend()
25 ax.set_xlim(2006, 2024)
26 ax.set_ylim(0, 100)
27 ax.set_xlabel("Year")
28 ax.set_ylabel("Average Score")
29 plt.savefig("score.pdf")
```

完成したscore.pyを実行してみましょう

- score.pdfが出力されたら，xdg-openで開いてみましょう
  - 下のような図が描けていればOK



練習問題

- score.pyを修正することで，以下のようなグラフを描いてみよう
  - 国語の受験者数(csvの4行目)の年度毎の推移をグラフに描いてみよう
  - 英語(readingとlistening)の受験者数(csvの18, 20行目)の年度毎の推移をグラフに描いてみよう
  - その他，csvファイルの項目を自由に選んで，その項目の年度推移をグラフに描いてみよう

作業が終わったら作業用ブランチを消す

- 実際のプログラム開発では, 作業用ブランチで新たに作成したファイル等もGitの管理下に置くが, この講義ではやりません
  - mainブランチに戻る（①）
  - exerciseブランチを削除（②）
  - 自分がいるブランチの確認（③）

```
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch -d exercise
Deleted branch exercise (was 2bf0644).
akiyama.shinichir.ga@icho01:~/compphys2/lecture2$ git branch
* main
```

①

②

③